Bus Servo ID Writing and Source Code Analysis

1. Servo Description

It is an automatic control system composed of a DC motor, reduction gears, a sensor, and control circuitry, which specifies the rotation angle of the output shaft by sending signals.

Differences between a servo and a motor:

A servo generally has a maximum rotation angle, while a DC motor rotates continuously. Therefore, a servo can only rotate within a certain angle and cannot rotate continuously like a DC motor. A regular DC motor cannot provide feedback on its rotation angle, whereas a servo can. (Digital servos can switch between servo mode and motor mode.)

2. Serial Bus Servo

A regular DC motor typically rotates continuously to provide power, while a servo is used to control the rotation of an object to a specific angle.

A bus servo is a derivative of a digital servo that communicates via an asynchronous serial bus, meaning it can be controlled by sending and receiving command packets, forming a closed-loop control system. Due to this feature, bus servos can be daisy-chained, making wiring simpler and reducing the use of serial ports. Because of this, when connecting bus servos, each servo's ID must be set in advance, or they cannot be individually controlled.

Bus servos use high-precision potentiometers internally, and unlike PWM servos, they can provide feedback on position, temperature, voltage, and other

information. Their accuracy and linearity are excellent, resulting in more stable robot operation and significantly extending the servo's lifespan.

Additionally, before assembling the servo, it must be set to its midpoint position. The servo midpoint refers to its initial position, considered the "zero point" from which it rotates to positive and negative angles. Therefore, the servo must be adjusted to the midpoint before being installed onto the servo horn.

This is because, during rotation, the moving parts drive the potentiometer. The software assumes the midpoint as the "zero point"; otherwise, the potentiometer might enter a "blind spot," causing the entire component to malfunction. Consequently, when using the robot, the specified angles may not be achieved, or the corresponding action sets may not match.

3. Procedure

1) We notice that the program defaults to setting the bus servo ID as 1, but this ID can be changed to other numbers.

```
29  void app_task_entry(void *argument)
30  {
31
32      /* 硬件初始化 */
33      serial_servo_set_id(0xFE, 1); //总线设置ID为1
34      serial_servo_set_position(1, HOLDER_MIN, 1000); //当总线舵机为1的时候，舵机进行转动到500位置，执行1秒
35      serial_servo_stop(1);//程序停止
36      // 循环  : RTOS任务中的循环，必须要有osDelay或者其他系统阻塞函数，否则会导致系统异常
37      for(;;) {
38          osDelay(300);
39          }
40  }
41
```

2) We can change the ID number to 5.

```
serial_servo_set_id(0xFE, 5); //总线设置ID为5
```

3) After recompiling and burning the program, connect the servo debugging board to the computer's serial port. Open the Bus Servo Terminal software

 , select the communication port, for example, COM15, click on parameter settings, and upon reading, it will show the ID number as 5, indicating that the ID has been successfully set.



2. 5ms ---------- 180 degrees

4. Code Analysis

File locations: Hiwonder/app.c

```
32    /* 硬件初始化 */
33    serial_servo_set_id(0xFE, 1); //总线设置ID为1
34    serial_servo_set_position(1, HOLDER_MIN, 1000); //当总线舵机为1的时候，舵机进行转动到500位置，执行1秒
35    serial_servo_stop(1);//程序停止
```

File locations: The function serial_servo_set_id() in Hiwonder/serial_servo.c is used to change the ID of a specific servo from old_id to new_id. The servo ID is sent to the servo via a serial data frame (defined by the array buf[]).

```
48  void serial_servo_set_id(uint8_t old_id, uint8_t new_id) {
49      uint8_t buf[7];
50      buf[0] = buf[1] = SERIAL_SERVO_FRAME_HEADER;
51      buf[2] = old_id;
52      buf[3] = 4;
53      buf[4] = SERIAL_SERVO_ID_WRITE;
54      buf[5] = new_id;
55      buf[6] = CheckSum(buf);
56      SerialWrite(buf, 7);
57  }
```

The function serial_servo_set_position() is used to set the position and duration of motion for the servo. Specific position and duration information is sent to the servo via a serial data frame (defined by the array buffer[]).

```
60  void serial_servo_set_position(uint8_t servo_id, uint16_t position, uint16_t duration)
61  {
62      if (servo_id > 31) {  //舵机ID不能大于31,可根据对应控制板修改
63          return;
64      }
65      uint8_t buffer[10];
66      position = position > 1000 ? 1000 : position;
67      buffer[0] = SERIAL_SERVO_FRAME_HEADER;
68      buffer[1] = SERIAL_SERVO_FRAME_HEADER;
69      buffer[2] = servo_id;
70      buffer[3] = 7;
71      buffer[4] = SERIAL_SERVO_MOVE_TIME_WRITE;
72      buffer[5] = GET_LOW_BYTE(position);
73      buffer[6] = GET_HIGH_BYTE(position);
74      buffer[7] = GET_LOW_BYTE(duration);
75      buffer[8] = GET_HIGH_BYTE(duration);
76      buffer[9] = CheckSum(buffer);
77      SerialWrite(buffer, 10);
78  }
```

The serial_servo_stop() function is designed to send a command to stop the servo via the serial port.

Execution process:

(1) Define a buffer to store the sent data frame.

(2) Fill the content of the data frame: frame header, servo ID, data length, stop servo, checksum, and finally send the data frame via the serial port.

```
118 void serial_servo_stop(uint8_t servo_id) {
119     uint8_t buffer[10];
120     buffer[0] = SERIAL_SERVO_FRAME_HEADER;
121     buffer[1] = SERIAL_SERVO_FRAME_HEADER;
122     buffer[2] = servo_id;
123     buffer[3] = 6;
124     buffer[4] = SERIAL_SERVO_MOVE_STOP;
125     buffer[5] = CheckSum(buffer);
126     SerialWrite(buffer, 6);
127 }
```